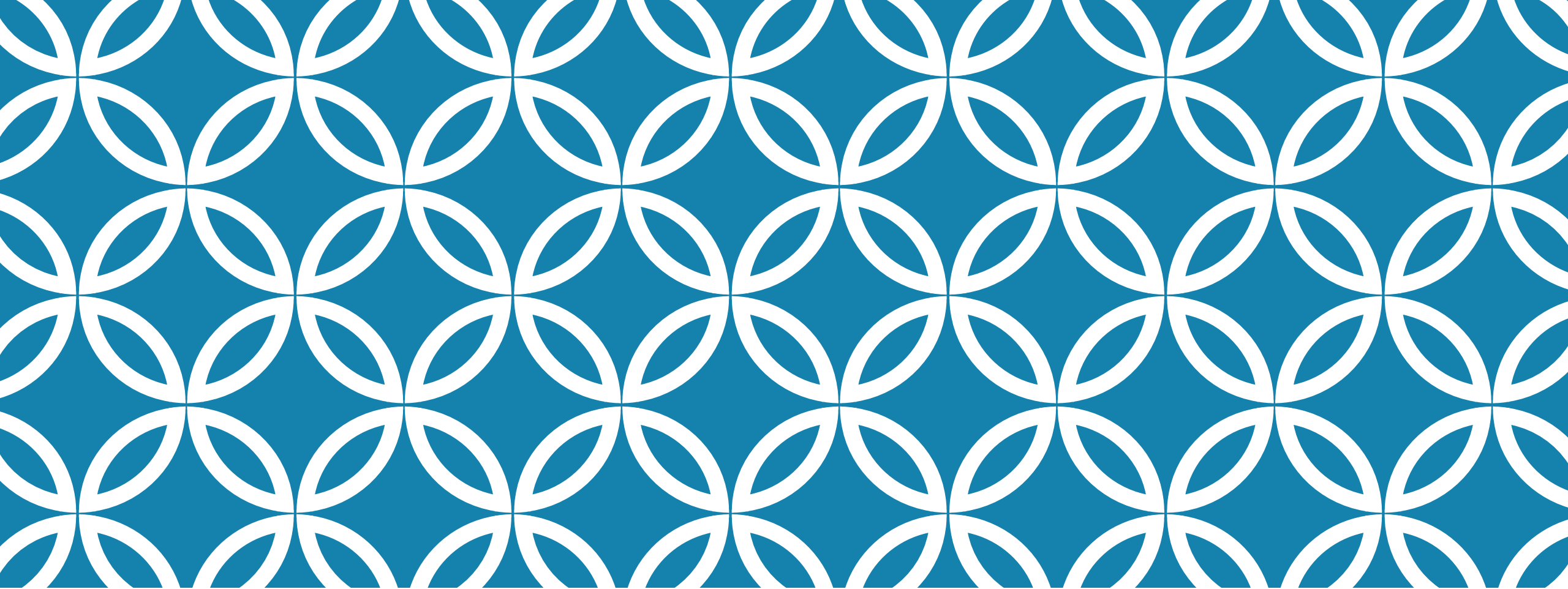




COMPUTER PROGRAMMING

WEEK 2

Instructor:

Nida Munawar

Lecturer

(Computer Science Department)

National University- FAST (KHI Campus)

ACKNOWLEDGMENT

- Publish material by Virtual University of Pakistan.
- Publish material by Deitel & Deitel.
- Publish material by Robert Lafore.



WEEK TWO, CLASS ONE

What is an Object?

OBJECT

Technical Definition:

- “An Object is an Instance of a class”
- “An Object is the implementation of a class”

In general we say :

“ Any tangible thing for which we want to save Information”

Now onwards we treat object technically.

SOME EXAMPLES

SHOES



Men sandal



Female sandal



Loafers



Pumpies

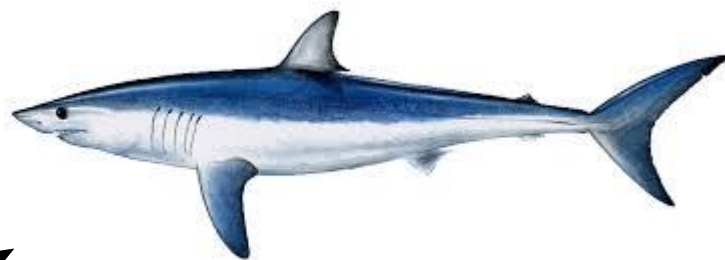


Formal



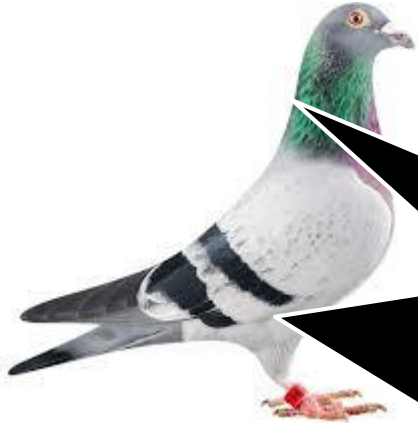
Humans





FISH





BIRD



DISCUSSION

- Object belongs to a group.
- Which similar.
- Have some common attributes.
- Have some common behaviors.
- We can categorized objects on some basic features ?



EXERCISE

Are there any 'objects' in this classroom?

EXERCISE

Are there any 'objects' in this classroom

- **Chair**
- **Multimedia**
- **Student**
- **Teacher**



EXERCISE

Possible Objects in this university?

CLASS

- Collection of Similar object.
- The objects that share some common features.
- It is the a design of an object.
- It is a detail of an object.
- It tell us what an object contains in it.

Technical Definition:

“ A class is blueprint of an object”

SUMMARIZE

A class :

- It's a blue print .
- It's a design or template.

An Object:

- Its an instance of a class.
- Implementation of a class.

NOTE: Classes are invisible, object are visible

CLASS

Accelerator Pedal

Brake Pedal

Steering Wheel



User-friendly “interfaces” to control the car

Their mechanism is *housed* inside the engineering drawings or blueprints

CLASS

Suppose you want to drive a car and make it go faster by pressing down on its accelerator pedal.

What must happen before you can do this?

CLASS

before you can drive a car, someone has to *design* it and *build* it.

A car typically begins as engineering drawings

Unfortunately, you cannot drive the engineering drawings of a car—before you can drive a car, it must be built from the engineering drawings that describe it

A completed car will have an actual accelerator pedal to make the car go faster. But even that's not

enough—the car will not accelerate on its own, so the driver must press the accelerator pedal to tell the car to go faster.

NOW LET'S USE OUR CAR EXAMPLE TO INTRODUCE THE KEY OBJECT-ORIENTED PROGRAMMING CONCEPTS

Performing a task in a program requires a function

The function hides from its user the complex tasks that it performs, just as the accelerator pedal of a car hides from the driver the complex mechanisms of making the car go Faster

In C++, we begin by creating a program unit called a class to house a function, just as a car's engineering drawings house the design of an accelerator pedal.

CLASS

Just as you cannot drive an engineering drawing of a car, you cannot “drive” a class.

Just as someone has to build a car from its engineering drawings before you can actually drive the car, *you must create an object of a class before you can get a program to perform the tasks the class describes.*

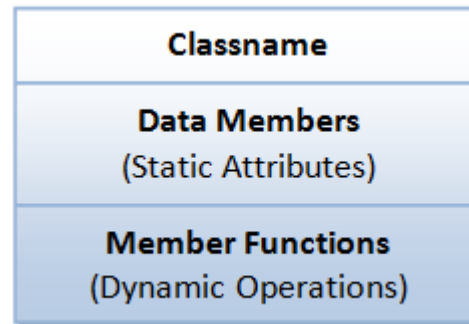
Note also that just as *many* cars can be built from the same engineering drawing, *many* objects can be built from the same class

A CLASS IS A 3-COMPARTMENT BOX ENCAPSULATING DATA AND FUNCTIONS

1. Classname (or identifier): identifies the class.

2. Data Members or **Variables** (or *attributes, states, fields*): contains the *attributes* of the class.

3. Member Functions (or *methods, behaviors, operations*): contains the *dynamic operations* of the class.



EXAMPLE OF CLASSES

Classname (Identifier)	Student	Circle
Data Member (Static attributes)	name grade	radius color
Member Functions (Dynamic Operations)	getName() printGrade()	getRadius() getArea()

EXAMPLE OF OBJECTS

Classname	<u>paul:Student</u>	<u>peter:Student</u>
Data Members	name="Paul Lee" grade=3.5	name="Peter Tan" grade=3.9
Member Functions	getName() printGrade()	getName() printGrade()

Two instances of the **Student** class

Each object of a class maintains its own copy of its attributes in memory

CLASSES

```
class Car
```

```
{
```

```
    void accelerate()
```

```
    { \\ logic for acceleration }
```

```
    void brake()
```

```
    { \\ logic for brakes }
```

```
};
```

CLASSES

```
class Car
```

```
{
```

```
    string model;  
    int numOfDoors;  
    string color;
```

```
    void accelerate()  
    { \\ logic for acceleration }
```

```
    void brake()  
    { \\ logic for brakes }
```

```
};
```

HOW TO USE THE CAR?

```
int main()
{
    Car car;
    car.accelerate();
}
```

GENERALIZED CLASS

- The class that only exhibits the common features of its objects.

Examples:

- ANIMAL
- BIRDS
- HUMAN
- No object of generalized class is found.

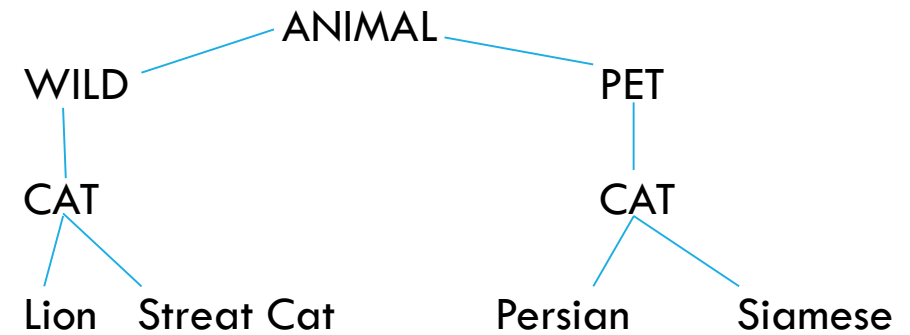
SPECIALIZED CLASS

- The class that exhibits different or unique features (behaviors)

ANIMAL (Generalized)

- Specialized:

- Mammals
- Cats
- Dog



TYPE OF CLASSES

1- Abstract Class: The classes we make against abstract concepts are called abstract classes. Abstract Classes can not exist standalone.

2- Concrete Class: The entities that actually we see in our real world are called concrete objects and classes made against these objects are called concrete classes.

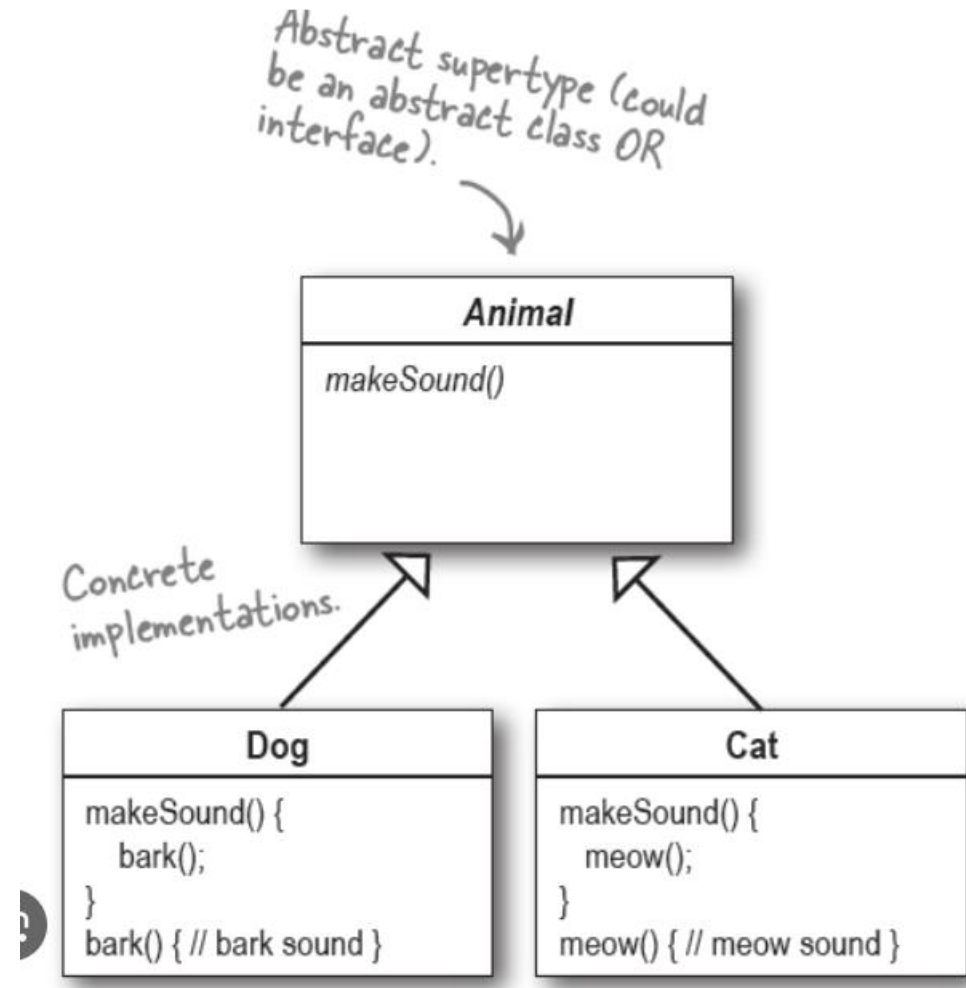
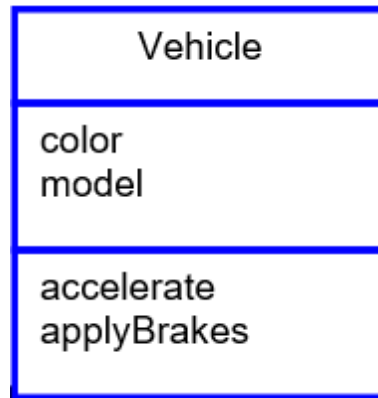
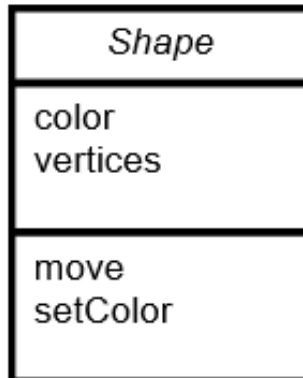
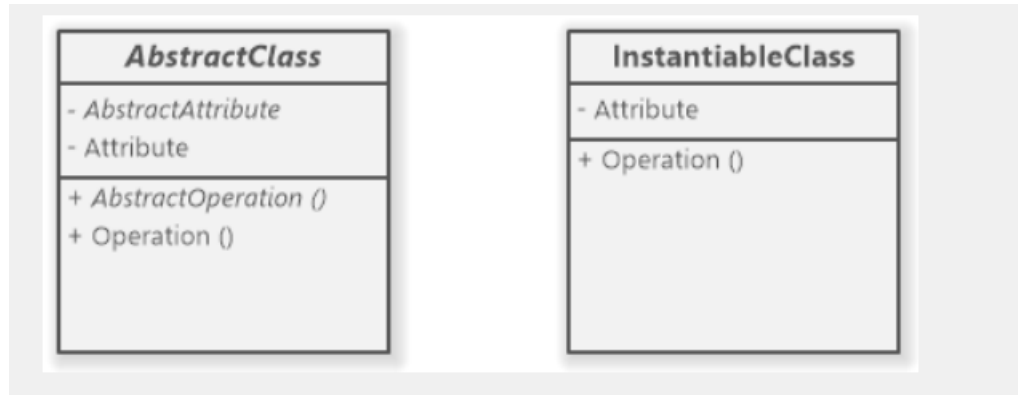
3- Sub-type: Sub-typing means that derived class is behaviorally compatible with the base class. Also known as Extension. Type B is called a subtype of type A if an instance of B can be used in every situation where an instance of A is required. In a way, the subtype does everything the supertype does and then some more, so it can be used as substitute for the “parent” or “supertype”.

4- Specialized class: Specialization means that derived class is behaviorally incompatible with the base class



While abstraction reduces complexity by hiding irrelevant detail, generalization reduces complexity by replacing multiple entities which perform similar functions with a single construct.

ABSTRACT CLASS



CONCRETE CLASS

Line	Circle	Triangle
color vertices length	color vertices radius	color vertices angle
move setColor getLength	move setColor computeArea	move setColor computeArea

Line is shape

Circle is a shape

Triangle is a shape

CLASS ACTIVITY



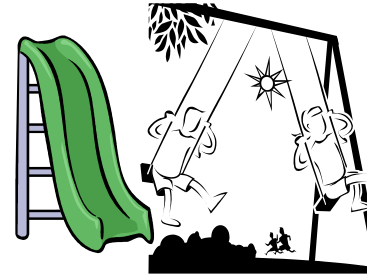
0/1- Library



2- Classroom



3- Parents



4- Playground



5- Pen



6- Book



7- Bag



8- Student

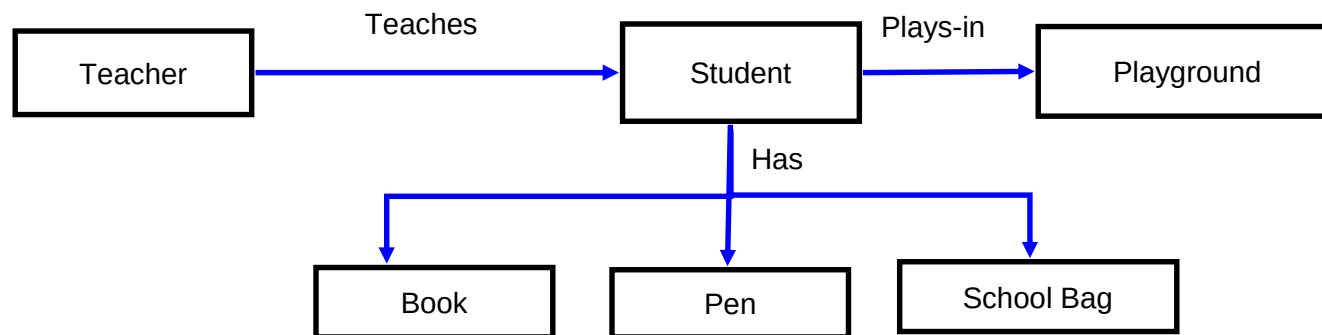


9 Teacher

QUESTIONS

- 1- Identify at least 3 behaviors and related data for an object.
- 2- Draw an object interaction model based on the objects .

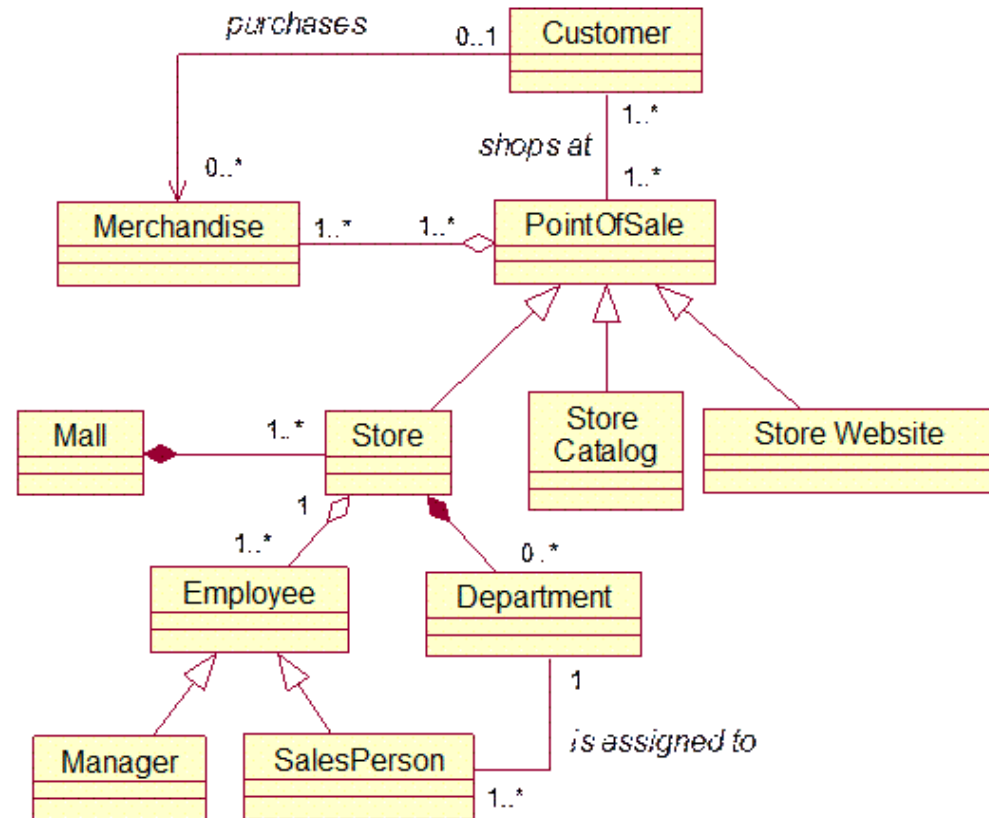
SOLUTION



QUESTION

Draw a class diagram that models the relationships between the following classes: *Mall*, *Store*, *Sales Person*, *Department*, *Manager*, *Merchandise*, *Store Catalog*, *Store Website*, and *Customer*.

SOLUTION





QUESTION

Identify the classes related to environment

ENVIRONMENT: UNIVERSITY

Salary Project
Doctor Classroom Grade
Department Library
Book Teacher
X-Ray White-board
Patient Course
Car Nurse Tree Employee
Testing Lab
Student Supervisor

ENVIRONMENT: **HOSPITAL**

Salary Project
Doctor Classroom Grade
Department Library
Book Teacher
X-Ray White-board
Patient Course
Car Nurse Tree Employee
Testing Lab
Student Supervisor

ENVIRONMENT: OFFICE

Salary Project
Doctor Classroom Grade
Department Library
Book Teacher
X-Ray White-board
Patient Course
Car Nurse Tree Employee
Testing Lab
Student Supervisor

ENVIRONMENT: **UNIVERSITY**

Write a program to enter record (Name, StudentID, CGPA & Warning Count) of 5 students with appropriate functions.

WEEK 3 LEC :1

CLASS VS STRUCT IN C++

Members of a class are **private** by default and members of struct are **public** by default

Classes can be inherited whereas structures cannot

Class is pass-by-reference and Struct is pass-by-copy, it means that, Class is a reference type and its object is created on the heap memory whereas structure is a value type and its object is created on the stack memory

A structure can't be abstract, a class can be

DIFFERENCE BETWEEN C STRUCTURES AND C++ STRUCTURES

Member functions inside structure: Structures in C cannot have member functions inside structure but Structures in C++ can have member functions along with data members.

Direct Initialization: We cannot directly initialize structure data members in C but we can do it in C++

Using struct keyword: In C, we need to use struct to declare a struct variable. In C++, struct is not necessary

Data Hiding: C structures do not allow concept of Data hiding but is permitted in C++ as C++ is an object oriented language whereas C is not.

ACCESS MODIFIERS

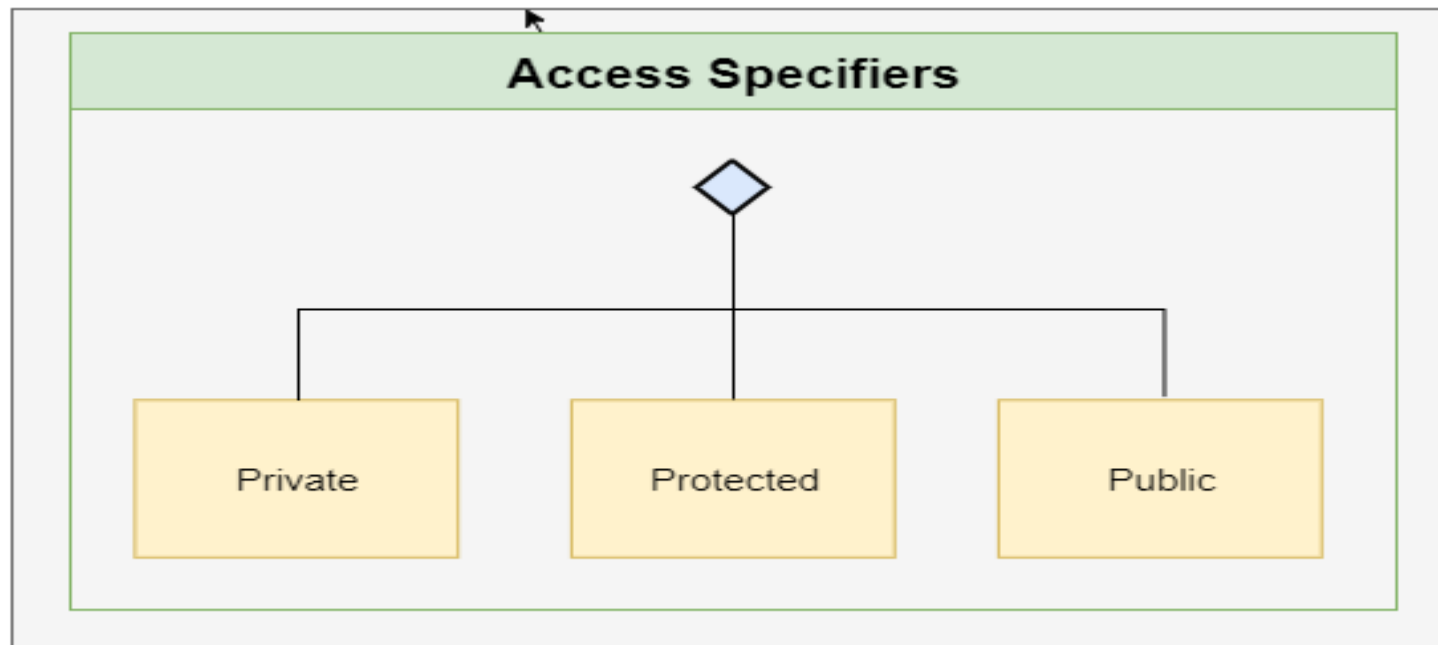
One of the main features of object-oriented programming languages such as C++ is **data hiding**.

Data hiding refers to restricting access to class members

Types of C++ Access Modifiers

In C++, there are 3 access modifiers:

- In C++, class members are considered **private** when no access modifier is used



PUBLIC ACCESS MODIFIER

- The `public` keyword is used to create public members (data and functions).
- The public members are accessible from any part of the program.

The public members of a class can be accessed from anywhere in the program using the direct member access operator (.) with the object of that class.

```
class MyClass { // The class
    public:      // Access specifier
    // class members goes here
};
```

PUBLIC ACCESS MODIFIER

```
class Car{  
    public:  
    void accelerate()  
    { \\ logic for acceleration }  
    void brake()  
    { \\ logic for brakes }  
};  
  
int main()  
{  
    Car car;  
    car.accelerate();  
}
```

PRIVATE ACCESS MODIFIER

The class members declared as *private* can be accessed only by the member functions inside the class. They are not allowed to be accessed directly by any object or function outside the class.

In C++, class members are considered **private** when no access modifier is used

PROTECTED ACCESS MODIFIER

Protected access modifier is similar to private access modifier in the sense that it can't be accessed outside , the difference is that the class members declared as Protected can be accessed by any subclass(derived class) of that class as well.

GETTER/SETTER FUNCTIONS

A class's private data members can be manipulated only by member functions of that Class. These member function names need not begin with *set* or *get*, but this naming convention is common

Getter functions (or accessor functions) are used to read value of a private member of some class

Setter functions (or mutator functions) are used to modify the value of a private member of some class

Set functions are also sometimes called **mutators** (because they mutate, or change, values), and *get* functions are also sometimes called **accessors** (because they access values).

Providing public *set* and *get* functions allows clients of a class to access the hidden data, but only *indirectly*

EXAMPLE

```
class BankAccount
{
    int accountNo;
    private:
        int PIN;

    public:
        int accountType;
}
```



```
class BankAccount
```

```
{
```

```
int PIN;
```

```
//private variable
```

```
public:
```

```
int get_PIN()
```

```
{
```

```
    return PIN;
```

```
}
```

```
}
```



```
class BankAccount
```

```
{
```

```
int accountNo; //private variable
```

```
public:
```

```
void set_accountNo(int num)
```

```
{
```

```
    accountNo = num;
```

```
}
```

```
}
```


DATA HIDING

Declaring data members with access specifier private is known as **data hiding**

Data hiding is a technique especially practiced in object-oriented programming (OOP). Data hiding is hiding the details of internal data members of an object.

Data hiding is also known as Information hiding

The use of getter & setter functions to read or modify class variables

ENCAPSULATION

Encapsulation is a process of capsulation of data and methods into a combined single unit.

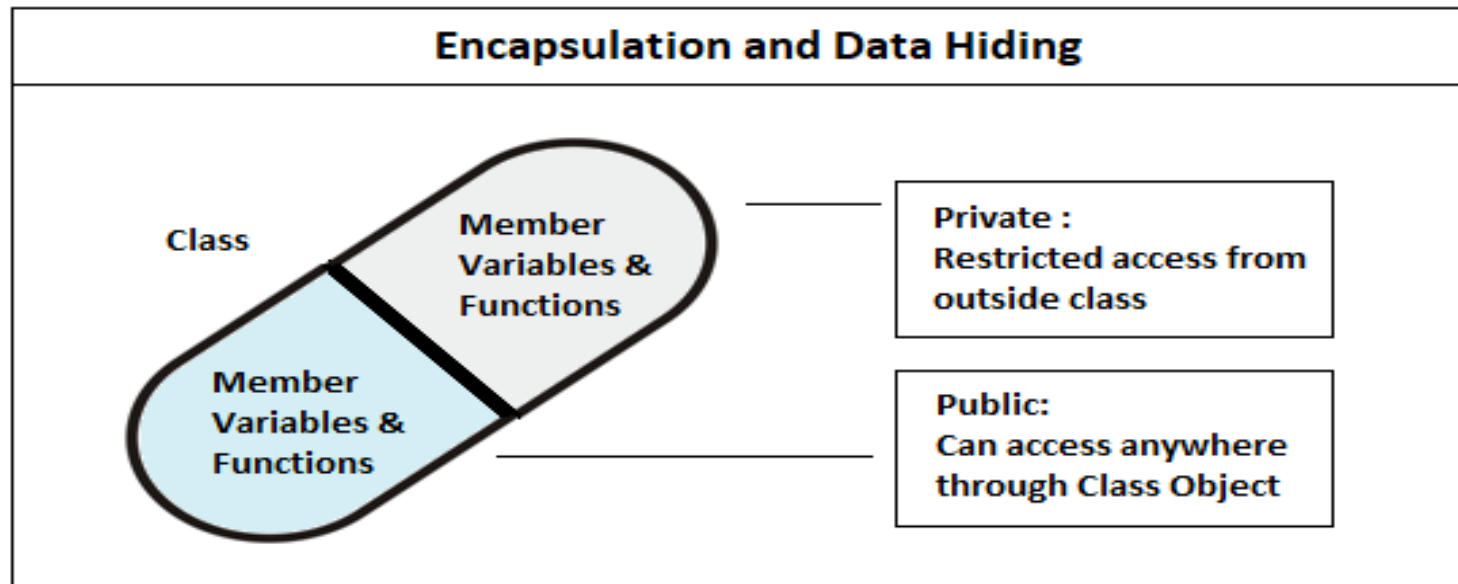
In C++, encapsulation is used along with the classes and access specifier concept.

Encapsulation is the process of combining data and function into a single unit called class.

Using the method of encapsulation the programmer cannot access the class directly.

You can assume it as a protective wrapper that stops random access of code defined outside that wrapper.

Encapsulation binds the data & functions together which keeps both safe from outside interference. Data encapsulation led to data hiding.





```
#include<iostream>
using namespace std;
```

```
class Encapsulation
{
    private:
        // data hidden from outside world
        int num;

    public:
        // function to set value of
        // variable x
        void set(int a)
        {
            num =a;
        }
}
```



```
} // function to return value of  
    // variable x
```

```
    int get()  
    {  
        return num;
```

```
};
```

```
// main function
```

```
int main()  
{  
    Encapsulation obj;
```

```
    obj.set(5);
```

```
    cout<<obj.get();
```

```
    return 0;
```

```
}
```

PLACING A CLASS IN A SEPARATE FILE FOR REUSABILITY

Header Files

Each of the previous examples consists of a single .cpp file, also known as a **source-code file**, that contains a class definition and a main function.

When building an object-oriented C++ program, it's customary to define reusable source code (such as a class) in a file that by convention has a .h filename extension—known as a **header file**.

Programs use #include preprocessor directives to include header files and take advantage of reusable software components, such as type string provided in the C++ Standard Library

GRADEBOOK.H

```
#include <iostream>

#include <string> // class GradeBook uses C++ standard string class
using namespace std;

class GradeBook{
public:
    void setCourseName( string name ) {
        courseName = name; }
    string getCourseName() {
        return courseName; }
    void displayMessage(){
        cout << "Welcome to the grade book for\n" << getCourseName()<< "!" << endl;
    }
private:
    string courseName; };
```

MAIN.CPP

```
#include <iostream>
#include "GradeBook.h" // include definition of class GradeBook
using namespace std;
int main(){
    GradeBook gradeBook1;
    gradeBook1.setCourseName("oop");
    cout << "gradeBook1 created for course: " << gradeBook1.getCourseName()<<
    endl;
} // end main
```


 C:\Users\Nida Munawar\Downloads\main.exe

```
gradeBook1 created for course: oop
```

```
-----
```

```
Process exited after 0.1743 seconds with return value 0
```

```
Press any key to continue . . .
```

PLACING A CLASS IN A SEPARATE FILE FOR REUSABILITY

A header file such as `GradeBook.h` cannot be used to begin program execution, because it does not contain a main function.

Line 2 instructs the C++ preprocessor to replace the directive with a copy of the contents of `GradeBook.h` (i.e., the `GradeBook` class definition) *before* the program is compiled.

When the source-code file `file.cpp` is compiled, it now contains the `GradeBook` class definition (because of the `#include`),

PLACING A CLASS IN A SEPARATE FILE FOR REUSABILITY

Notice that the name of the GradeBook.h header file in line 4 is enclosed in quotes (" ") rather than angle brackets (< >). Normally, a program's source-code files and

user-defined header files are placed in the same directory. When the preprocessor encounters a header file name in quotes, it attempts to locate the header file in the same directory as the file in which the #include directive appears

DATA VALIDATION/VALIDITY CHECKING

Data validation is performed to ensure that class members are provided data in “*correct format*”

Validation rules vary according to requirements

Where to perform *validation*?

DATA VALIDATION/VALIDITY CHECKING

```
void setCourseName(string name) // a setter function
{
    if ( name.length() <= 25 )
        courseName = name;
    if ( name.length() > 25 )
    {
        courseName = name.substr( 0, 25 );
        cout << "Name exceeds max length (25)" << endl;
    }
}
```

SEPARATING INTERFACE FROM IMPLEMENTATION

It's a good practice to define member functions *outside* class definition

Implementation details can be hidden from client code

Do not write client code that depends on class' implementation

SEPARATING INTERFACE FROM IMPLEMENTATION

```
// A.h  
  
class A{  
    public:  
    int a;  
    void getA();  
  
};
```

SEPARATING INTERFACE FROM IMPLEMENTATION

```
//B.cpp
```

```
#include <iostream>
```

```
#include "A.h"
```

```
using namespace std;
```

```
void A::getA()
```

```
{
```

```
    a = 10;
```

```
    cout << a;
```

```
}
```


SEPARATING INTERFACE FROM IMPLEMENTATION

```
//Test.cpp
```

```
//place all files within same project
```

```
#include "A.h"
```

```
int main(){
```

```
    A c;
```

```
    c.getA();
```

```
}
```

EXAMPLE: BANK ACCOUNT

Create an Account class to represent customers' bank accounts. Include a data member of type int to represent the account balance.

You need to initialize the data members. But while doing so, you should validate the initial balance to ensure that it's greater than or equal to 0. If not, set the balance to 0 and display an error message indicating that the initial balance was invalid.

Provide three member functions. Member function credit should add an amount to the current balance. Member function debit should withdraw money from the Account and ensure that the debit amount does not exceed the Account's balance. If it does, the balance should be left unchanged and the function should print a message indicating "Debit amount exceeded account balance." Member function getBalance() should return the current balance. Create a program that creates two Account objects and tests the member functions of class Account.

EXERCISE

A bank wants a simple application module to manage the accounts of its customers. For every new customer, the app must let us fill in the details including his Name, Age, NIC#, Address, Opening Balance, Current Balance, Contact# & PIN. These details may be modified later except for the PIN. At any given time, the customer can check his balance.

Also, tax must be calculated (Tax is 0.15% of the current balance for customers aged 60 or above and 0.25% for all other customers).